

UNIT 3 · WEEKS 3–4

Files and file systems

📅 Weeks 3–4 · dates to be confirmed </> Lab TD2 (solved + AI challenge) ~ 4 animations

MANUAL M3 · THEORY OF THE UNIT

Files and file systems

Core reading for unit 3 (≈ 1.5 h). Every section ends with a **Practice this** → link. The outputs are real: they were obtained with the TD2 programs on the course machine (Ubuntu 26.04, Linux 7.0, ext4 on NVMe).

CONTENTS

- | | | | |
|----|-------------------------------------|----|--|
| §1 | Two levels: the disk and the kernel | §5 | Special files: FIFOs, devices, <code>/proc</code> |
| §2 | The inode: metadata and blocks | §6 | Walking directories |
| §3 | Names: directories, links and paths | §7 | Beyond: journaling, copy-on-write and <code>fsync</code> |
| §4 | Open files: the three tables | | |

§1 Two levels: the disk and the kernel

A file exists in two places. **On disk**, the file system organises a huge array of blocks (4 KiB on the course machine): a *superblock* with the general description, bitmaps that say which blocks and inodes are free, the *inode tables* and the *data blocks*. ext4 repeats this structure in *block groups* along the partition, with superblock copies. The number of inodes is fixed at format time:

Real output

```
$ df -iT . | cut -c1-80; stat -f -c 'block size: %S bytes' .
Filesystem      Type  Inodes   IUsed   IFree IUse% Mounted on
/dev/nvme0n1p5 ext4 25600000 2372366 23227634   10% /
block size: 4096 bytes
```

In the kernel, while a process uses the file, there are three tables: each process's descriptor table, the system's open file table and the in-memory inode table (§4). On top of everything, the *page cache* keeps recently read blocks in RAM: that is why the second read of a file almost never touches the disk.

See it in slides P3 →

§2 The inode: metadata and blocks

Unix separates a file's **content** from its **metadata**. The metadata lives in an **inode**, identified by a number: type and permissions (`st_mode`), number of hard links (`st_nlink`), owner and group, size, device, timestamps and the list of blocks. The inode does **not** contain the name. `stat` , `fstat` (on a descriptor) and `lstat` (without following links) read it:

Real output

```
$ stat notas.txt
  File: notas.txt
  size: 11          Blocks: 8          IO Block: 4096   regular file
Device: 259,5      Inode: 12190686   Links: 2
Access: (0664/-rw-rw-r--)  Uid: ( 1000/student)   Gid: ( 1000/student)
Access: 2026-09-30 13:12:49.872600645 -0500
Modify: 2026-09-30 13:12:49.872600645 -0500
Change: 2026-09-30 13:12:49.879601063 -0500
 Birth: 2026-09-30 13:12:49.872600645 -0500
```

The four timestamps: **Access** (last read; Linux updates it lazily with `relatime`), **Modify** (last change of the content), **Change** (last change of the *inode*: permissions, links, owner... or content) and **Birth** (creation, available on ext4 and read with `statx`). It is a frequent mistake —the original slides made it— to think `ctime` is creation.

The type is tested with the `S_ISREG` , `S_ISDIR` , `S_ISLNK` , `S_ISCHR` , `S_ISBLK` , `S_ISFIFO` and `S_ISSOCK` macros on `st_mode` . The typical errors of `stat` are `ENOENT` (does not exist), `EACCES` (missing search permission `x` on a directory of the path), `ENOTDIR` and `ELOOP` .

Where are the blocks? ext2 and ext3 stored 15 pointers in the inode: 12 direct, one single indirect (a block full of pointers), one double and one triple. With 4 KiB blocks and 4-byte pointers, 1024 fit per block and the scheme reaches about 4 TiB, but a distant offset requires reading up to 3 index blocks. ext4 uses **extents**: contiguous runs described by (logical start, length ≤ 32,768 blocks, physical start). Four fit in the inode; if more are needed, a tree is built. The 10 MB file written by TD2 takes only two:

Real output

```
$ head -c 10000000 /dev/urandom > big.bin; sync big.bin; filefrag -v big.bin
Filesystem type is: ef53
File size of big.bin is 10000000 (2442 blocks of 4096 bytes)
ext:      logical_offset:      physical_offset: length:   expected: flags:
  0:         0..      2047:   15654912..   15656959:   2048:
  1:      2048..      2441:   15673843..   15674236:    394:   15656960: last,eof
big.bin: 2 extents found
```

Practice this → A3.4 From the inode to the blocks · TD2 · 1

§3 Names: directories, links and paths

A **directory** is a file whose data is a list of *name* → *inode number* pairs, plus `.` (itself) and `..` (its parent). To open `/home/student/notas.txt` the kernel starts at the root inode (number 2 on ext4), reads the directory, looks up `home`, continues with its inode, and so on component by component (*path_resolution(7)*). Every component requires search permission (`x`) on the directory. Linux keeps recent resolutions in the *dentry cache*.

A **hard link** (`link`, `ln`) is another entry pointing to the **same** inode: both names are equally "original" and `st_nlink` counts them. They are not allowed to directories (they would break the tree) nor across file systems (inode numbers are local). A **symbolic link** (`symlink`, `ln -s`) is a new inode whose content is a *path*; resolution follows it (at most 40 times: after that, `ELOOP`) and it can dangle if the target disappears.

Real output

```
$ ln notas.txt copia.txt; ln -s notas.txt atajo; ls -li notas.txt copia.txt atajo
12190687 lrwxrwxrwx 1 student student 9 Sep 30 13:12 atajo → notas.txt
12190686 -rw-rw-r-- 2 student student 11 Sep 30 13:12 copia.txt
12190686 -rw-rw-r-- 2 student student 11 Sep 30 13:12 notas.txt
```

`rm` calls `unlink`, which removes an **entry** and subtracts 1 from `st_nlink`. The kernel frees the inode and its blocks only when `st_nlink = 0` **and** no process has it open. Meanwhile the file is still readable through its descriptor:

Real output

```
$ exec 3< notas.txt; rm notas.txt copia.txt; readlink /proc/$$/fd/3; cat <&3; ls -l
atajo; cat atajo
~/fso/tds/tests/tmp/td2/notas.txt (deleted)
hola inodo
lrwxrwxrwx 1 student student 9 Sep 30 13:12 atajo → notas.txt
cat: atajo: No such file or directory
[exit status 1]
```

⚠ Check then use: the TOCTOU race

if (`access(path, W_OK) == 0`) `fd = open(path, O_WRONLY)`; has a window: between the two calls another process can replace `path` with a link to another file. Open directly and handle the error (`EACCES`, `ENOENT`), or use `O_NOFOLLOW` or `openat2()` with `RESOLVE_NO_SYMLINKS`. Besides, `access` checks with the *real* UID, not the effective one.

Practice this → A3.1 Inodes and directories

§4 Open files: the three tables

`open(path, flags, mode)` resolves the path, checks permissions and returns the **lowest free descriptor**. The flags: access (`O_RDONLY`, `O_WRONLY`, `O_RDWR`), creation (`O_CREAT`, `O_EXCL`, `O_TRUNC` ; the requested permissions go through the `umask`) and status (`O_APPEND`, `O_NONBLOCK`, `O_CLOEXEC`). glibc implements `open` with `openat(AT_FDCWD, ...)`, which is what `strace` shows.

Behind every descriptor there are three levels:

1. **Descriptor table**, one per process: the number (0, 1, 2, 3...) is an index into it.
2. **Open file table**, one for the whole system: every `open` creates an entry with the mode, the current **offset** and a reference counter.
3. **In-memory inode table**: one copy of the inode per file, shared by all the openings.

That is why it matters who shares each entry. Two `open` calls on the same file create two entries with independent offsets; `dup`, `dup2` and `fork` copy the *pointer* and share the offset. With `donnees.txt = abcdefgh`:

Real output

```
$ ./alias_open
abcdabcd
```

Real output

```
$ ./alias_dup
abcdefgh
```

Practical consequences: `./prog > out 2> out` opens `out` twice and the error output overwrites the standard output; `./prog > out 2>&1` uses `dup2` and loses nothing. With `O_APPEND`, every `write` moves to the end atomically: that is how servers write their logs. `close` subtracts one reference; the entry is freed when it reaches 0.

Practice this → A3.2 The three tables · TD2 · 3

§5 Special files: FIFOs, devices, `/proc`

"Everything is a file" means that many things that are not data on a disk use the same `open` / `read` / `write` / `close` interface. A **FIFO** (named pipe, `mkfifo`) is an inode without blocks: what one process writes, another reads through a 64 KiB kernel buffer. Its rules, which the TD2 chat uses:

- `open(O_RDONLY)` waits until someone opens for writing, and vice versa (with `O_NONBLOCK`, a writer with no reader gets `ENXIO`).
- A write of up to `PIPE_BUF` = 4096 bytes is **atomic**: it does not mix with another writer's. A larger one may mix.
- `read` returns 0 (end of file) when the **last** writer closes.

- Writing with no reader at all produces the `SIGPIPE` signal (or the `EPIPE` error if it is ignored). Other special files: character and block **devices** in `/dev` (created with `mknod`), UNIX domain **sockets** (unit 8 and TD8) and the virtual file systems `/proc` and `/sys`, whose "files" the kernel generates when they are read.

Real output

```
$ ./fifo_server chat.fifo & sleep 0.3; echo 'hola desde el cliente' | ./fifo_client
chat.fifo; wait
server: waiting for clients on chat.fifo
hola desde el cliente
```

Practice this → A3.3 FIFO: the chat · TD2 · 2

§6 Walking directories

`opendir` opens a directory, `readdir` returns one `struct dirent` per entry (or `NULL` at the end) and `closedir` closes it. The entries come in the file system's internal order —on ext4, a hash index order—, and include `.` and `..`. From `struct dirent`, `d_name`, `d_ino` and `d_type` matter; the latter saves a `stat`, but it may be `DT_UNKNOWN` on some file systems, and then `lstat` is needed. When walking a tree, symbolic links to directories are never followed: they could form cycles.

C tds/td2/myls.c

```
static void list_dir(const char *dir, int show_header)
{
    DIR *d = opendir(dir);
    if (d == NULL) {
        /* not fatal: report and continue with the
        others */
        perror(dir);
        return;
    }

    /* Collect the names so that we can sort them. */
    size_t n = 0, cap = 16;
    char **names = malloc(cap * sizeof *names);
    exit_if(names == NULL, "malloc");
    struct dirent *e;
    while ((e = readdir(d)) != NULL) {
        if (n == cap) {
            cap *= 2;
            names = realloc(names, cap * sizeof *names);
            exit_if(names == NULL, "realloc");
        }
        names[n] = strdup(e->d_name);
        exit_if(names[n] == NULL, "strdup");
        n++;
    }
    exit_if(closedir(d) == -1, "closedir");
}
```

Real output

```
$ ./mys -lR demo
demo:
drwxrwxr-x   3 student student   4096 2026-09-30 13:12 .
drwxrwxr-x   3 student student   4096 2026-09-30 13:12 ..
-rw-rw-r--   1 student student     3 2026-09-30 13:12 a.txt
lrwxrwxrwx   1 student student     5 2026-09-30 13:12 link → a.txt
prw-rw-r--   1 student student     0 2026-09-30 13:12 pipe
drwxrwxr-x   2 student student   4096 2026-09-30 13:12 sub

demo/sub:
drwxrwxr-x   2 student student   4096 2026-09-30 13:12 .
drwxrwxr-x   3 student student   4096 2026-09-30 13:12 ..
```

Practice this → TD2 · 4 Your own 1s

§7 Beyond: journaling, copy-on-write and `fsync`

- **Journaling.** Creating a file modifies several blocks (bitmap, inode, directory). If the power fails half-way, the file system is left inconsistent. ext4 and XFS first write the operation to a journal and then apply it: on reboot it is enough to replay or discard what was pending.
- **Copy-on-write.** Btrfs and ZFS never overwrite a block in place: they write a new one and change the pointer. That gives almost free snapshots and checksums of every block.
- **Page cache and `fsync`.** `write` returns when the data is in the kernel cache, not on the disk. A program that needs durability (a database, an editor saving) calls `fsync`, and for the *name* to be durable too, `fsync` on the directory.
- **FUSE.** A file system can be implemented as a user process (sshfs, rclone): the kernel forwards the calls to it.